

Class 03: MicroCPU 블록 설계

건양대학교 국방반도체공학과
백종섭 교수

- 📖 3.1 cpu_pkg — opcode_t, state_t enum 정의를 읽고 설명할 수 있다
- 📖 3.2 sysclk — clock gating과 2분주 구조를 코드로 구현할 수 있다
- 📖 3.3 mem — 동기 메모리의 읽기/쓰기 동작을 코드로 구현할 수 있다
- 📖 3.4 Controller — Moore FSM의 상태 천이와 출력 생성을 코드로 구현할 수 있다
- 📖 3.5 IR — 명령어 래치와 필드 분리를 코드로 구현할 수 있다
- 📖 3.6 Register File — 2R+1W 레지스터 파일을 코드로 구현할 수 있다
- 📖 3.7 op_mux — 피연산자 선택 MUX를 코드로 구현할 수 있다
- 📖 3.8 PC — load/enable 우선순위를 가진 카운터를 코드로 구현할 수 있다
- 📖 3.9 addr_mux — 주소 선택 MUX를 코드로 구현할 수 있다
- 📖 3.10 ALU — 산술/논리 연산기와 zero 플래그를 코드로 구현할 수 있다

opcode_t와 state_t를 enum으로 정의하여 모든 블록이 공유한다

- **opcode_t** (3비트): 8개 명령어
 - 제어: WFR
 - 분기: BRZ, BRA
 - 데이터 이동: LDA, STA
 - 산술/논리: ADD, AND, NOT
- **state_t** (3비트): 8개 FSM 상태
 - Fetch: S0~S2
 - Decode: S3
 - Execute: S4~S7
- `import cpu_pkg::*;` 로 모든 모듈에서 사용

```
package cpu_pkg;

typedef enum logic [2:0] {
    WFR,           // Control
    BRZ, BRA,      // Branch
    LDA, STA,      // Data Move
    ADD, AND, NOT  // ALU
} opcode_t;

typedef enum logic [2:0] {
    INST_ADDR,     // S0 Fetch
    INST_FETCH,    // S1
    INST_LOAD,     // S2
    INST_DECODE,   // S3
    OP_ADDR,       // S4 Execute
    OP_FETCH,      // S5
    OP_ALU,        // S6
    UPDATE         // S7
} state_t;

endpackage : cpu_pkg
```

clk_ext를 입력받아 내부 클럭 clk_sys를 생성한다. halt 시 정지한다

포트	방향	폭	From	설명
clk_ext	in	1	외부 핀	외부 클럭
rst_n	in	1	외부 핀	비동기 리셋
halt	in	1	Controller	High이면 clk_sys 정지
clk_sys	out	1		2분주된 내부 클럭

```
wire clk_i = clk_ext & ~halt;

always_ff @(posedge clk_i or negedge rst_n)
    if (!rst_n) div <= 1'b0;
    else      div <= ~div;

assign clk_sys = div;
```

addr를 입력받아 명령어 또는 데이터를 읽고 쓴다

포트	방향	폭	From	설명
clk	in	1	sysclk	시스템 클럭
read	in	1	Controller	High이면 addr의 데이터를 읽음
write	in	1	Controller	High이면 addr에 데이터를 씀
addr	in	8	addr_mux	읽기/쓰기 대상 메모리 주소
data_in	in	16	regfile	쓰기 데이터
data_out	out	16		읽은 데이터

```
logic [15:0] memory [0:255];

// Write
always @(posedge clk)
    if (write && !read)
        memory[addr] <= data_in;

// Read
always_ff @(posedge clk)
    if (read && !write)
        data_out <= memory[addr];
```

8-상태 FSM이 opcode와 zero를 입력받아 9개 제어 신호를 생성한다

포트	방향	폭	From	설명
clk	in	1	sysclk	시스템 클럭
rst_n	in	1	외부 핀	비동기 리셋
ir_opcode	in	3	IR	명령어의 opcode
zero	in	1	ALU	ALU zero 플래그
fetch_phase	out	1		Fetch/Execute phase 표시
mem_rd	out	1		MEM 읽기 enable
mem_wr	out	1		MEM 쓰기 enable
ir_load	out	1		IR 래치 enable
load_reg	out	1		regfile 쓰기 enable
inc_pc	out	1		PC 증가 enable
load_pc	out	1		PC 분기 주소 로드 enable
halt	out	1		WFR 시 High로 래치

```
// 1. opcode decode (조합)
assign is_op_memrd = (ir_opcode inside {ADD, AND, LDA});
assign is_not = (ir_opcode == NOT);
assign is_wfr = (ir_opcode == WFR);
assign is_brz = (ir_opcode == BRZ);
assign is_bra = (ir_opcode == BRA);
assign is_sta = (ir_opcode == STA);

// 2. state FF
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) state <= INST_ADDR;
    else if (!halt) state <= state.next();

// 3. 출력 FF - next-state 기반
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin /* 초기화 */ end
    else if (!halt) begin
        fetch_phase <= ...; // next-state 기반
        case (state.next())
            // Fetch:
            //     mem_rd, ir_load (고정)
            // Execute:
            //     inc_pc, halt(is_wfr)
            //     mem_rd(is_op_memrd), mem_wr(is_sta)
            //     load_reg(is_op_memrd|is_not)
            //     inc_pc(is_brz&&zero), load_pc(is_bra)
        endcase
    end
```

메모리에서 읽은 16비트 명령어를 래치하고, 5개 필드로 분리하여 각 블록에 전달한다

포트	방향	폭	From	설명
clk	in	1	sysclk	시스템 클럭
rst_n	in	1	외부 핀	비동기 리셋
enable	in	1	Controller	ir_load
din	in	16	MEM	래치할 명령어
ir_opcode	out	3		din[15:13] 명령어 코드
ir_mode	out	1		din[12] 주소 지정 방식
ir_rd	out	2		din[11:10] 목적 레지스터 주소
ir_rs	out	2		din[9:8] 소스 레지스터 주소
ir_addr	out	8		din[7:0] 메모리 주소

```

always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
        ir_opcode <= WFR;
        ir_mode   <= 1'b0;
        ir_rd     <= 2'b0;
        ir_rs     <= 2'b0;
        ir_addr   <= 8'b0;
    end
    else if (enable) begin
        ir_opcode <= opcode_t'(din[15:13]);
        ir_mode   <= din[12];
        ir_rd     <= din[11:10];
        ir_rs     <= din[9:8];
        ir_addr   <= din[7:0];
    end
end
    
```

rd, rs 주소로 레지스터를 선택하고, ALU에 Rd/Rs 값을 공급하며 연산 결과를 Rd에 저장한다

포트	방향	폭	From	설명
clk	in	1	sysclk	시스템 클럭
rst_n	in	1	외부 핀	비동기 리셋
rd_addr	in	2	IR	Rd 읽기 주소
rs_addr	in	2	IR	Rs 읽기 주소
wr_data	in	16	ALU	쓰기 데이터
wr_addr	in	2	IR	쓰기 주소
wr_en	in	1	Controller	load_reg
rd_data	out	16		regs[rd_addr] 조합 출력
rs_data	out	16		regs[rs_addr] 조합 출력

```

logic [15:0] regs [0:3];

// Synchronous write
always_ff @(posedge clk, negedge rst_n)
    if (!rst_n) begin
        regs[0] <= '0;
        regs[1] <= '0;
        regs[2] <= '0;
        regs[3] <= '0;
    end
    else if (wr_en)
        regs[wr_addr] <= wr_data;

// Combinational read
assign rd_data = regs[rd_addr];
assign rs_data = regs[rs_addr];
    
```


mode에 따라 메모리 값(data_out) 또는 레지스터 값(rs_data)을 선택하여 ALU에 전달한다

포트	방향	폭	From	설명
din_a	in	16	MEM	첫 번째 입력
din_b	in	16	regfile	두 번째 입력
sel_a	in	1	IR	High이면 din_a, Low이면 din_b 선택
dout	out	16		선택된 출력

```
// mux2to1 #(16)
assign dout = sel_a ? din_a : din_b;
```

다음 명령어의 메모리 주소를 가리킨다. 매 명령어 후 자동 증가하고, BRA 시 ir_addr를 로드한다

포트	방향	폭	From	설명
clk	in	1	sysclk	시스템 클럭
rst_n	in	1	외부 핀	비동기 리셋
load	in	1	Controller	load_pc High이면 din을 로드 enable보다 우선
enable	in	1	Controller	inc_pc High이면 pc_count += 1
din	in	8	IR	load 시 로드할 주소 값
pc_count	out	8		갱신된 PC 값

```
always_ff @(posedge clk, negedge rst_n)
  if (!rst_n)      pc_count <= '0;
  else if (load)    pc_count <= din;
  else if (enable)  pc_count <= pc_count + 1;
```

Fetch 시 pc_addr, Execute 시 ir_addr을 선택하여 MEM에 전달한다

포트	방향	폭	From	설명
din_a	in	8	PC	첫 번째 입력. sel_a가 High이면 선택
din_b	in	8	IR	두 번째 입력. sel_a가 Low이면 선택
sel_a	in	1	Controller	1이면 din_a, 0이면 din_b 선택
dout	out	8		선택된 출력

```
// mux2to1 #(8)
assign dout = sel_a ? din_a : din_b;
```

Rd와 피연산자로 ADD, AND, NOT, LDA 연산을 수행하고, zero 플래그를 출력한다

포트	방향	폭	From	설명
accum	in	16	regfile	첫 번째 피연산자
din	in	16	op_mux	두 번째 피연산자
opcode	in	3	IR	연산 종류 선택
dout	out	16		연산 결과
zero	out	1		~(dout). 연산 결과가 0이면 High

```
always_comb
  unique case (opcode)
    ADD : dout = accum + din;
    AND : dout = accum & din;
    NOT : dout = ~accum;
    LDA : dout = din;
    default : dout = accum;
  endcase

  assign zero = ~(|dout);
```